

RIT — Plataforma de Monitoramento de Desempenho

Documentação do processo de desenvolvimento — Case Técnico (Software Analyst)

Nicolas da Silva Maciel

Introdução

Escrevi este documento durante todo o desenvolvimento do projeto para a case para mostrar meus pensamentos e tomadas de decisões, deixo abaixo o link dos projetos no Figma e do repositório GitHub!

Projeto no Figma: <https://www.figma.com/design/lWaHHOTTVsoV4ojSUDGi1G/Untitled?node-id=0-1&t=XYOBgHYoY5ReSecs-1>

Repositório no GitHub: <https://github.com/nickkjj/Rit-App>

1. Ideia inicial e modelagem do banco de dados

A minha ideia inicial foi de fazer um painel dashboard que torne possível o monitoramento de desempenho dos funcionários.

Observei o arquivo .sql, que descreve como devem ser a tabela de funcionários e a de liderança, mas notei que ainda era necessário criar uma forma de avaliar os usuários.

Para isso, adicionei as seguintes tabelas para questões e avaliações:

```
CREATE TABLE questions_version (  
  id SERIAL PRIMARY KEY,  
  created_at TIMESTAMP,  
  created_by INTEGER REFERENCES employee(id),  
  status BOOLEAN  
);
```

```
CREATE TABLE question (  
  id SERIAL PRIMARY KEY,  
  questions_version_id INTEGER REFERENCES questions_version(id),  
  title TEXT,  
  weight INTEGER  
);
```

```
CREATE TABLE rating (  
  id SERIAL PRIMARY KEY,  
  created_by INTEGER REFERENCES employee(id),  
  employee_id INTEGER REFERENCES employee(id),  
  created_at TIMESTAMP  
);
```

```
CREATE TABLE answer (  
  id SERIAL PRIMARY KEY,  
  question_id INTEGER REFERENCES question(id),  
  rating_id INTEGER REFERENCES rating(id),  
  answer INTEGER  
);
```

Dessa forma, futuramente, caso seja necessário, tornamos possível a alteração das perguntas de avaliação e mantemos o histórico salvo para monitoramento.

OBS: A demanda de processamento na busca de liderados indiretos e diretos em cada página aumenta conforme a quantidade de usuários cadastrados. Por isso, cogitei criar uma tabela de fechamento (closure), onde todos esses processamentos ocorreriam apenas uma vez, sempre que ocorresse uma alteração no banco. Mas, como se trata de uma case, não existem muitos cadastros de usuários, além de que, em uma situação real, a manutenção se torna complexa demais com uma tabela de fechamento (o banco precisaria de scripts pesados apagando e criando relações sempre que um usuário é cadastrado ou tem sua liderança ou liderado trocados). Dessa forma, optei por utilizar o Postgres, que nativamente possui a consulta recursiva (CTE), que varre a árvore de hierarquia e não necessita da criação de novas tabelas para se manter atualizada.

Segue um diagrama representando as tabelas do banco a ser criado:

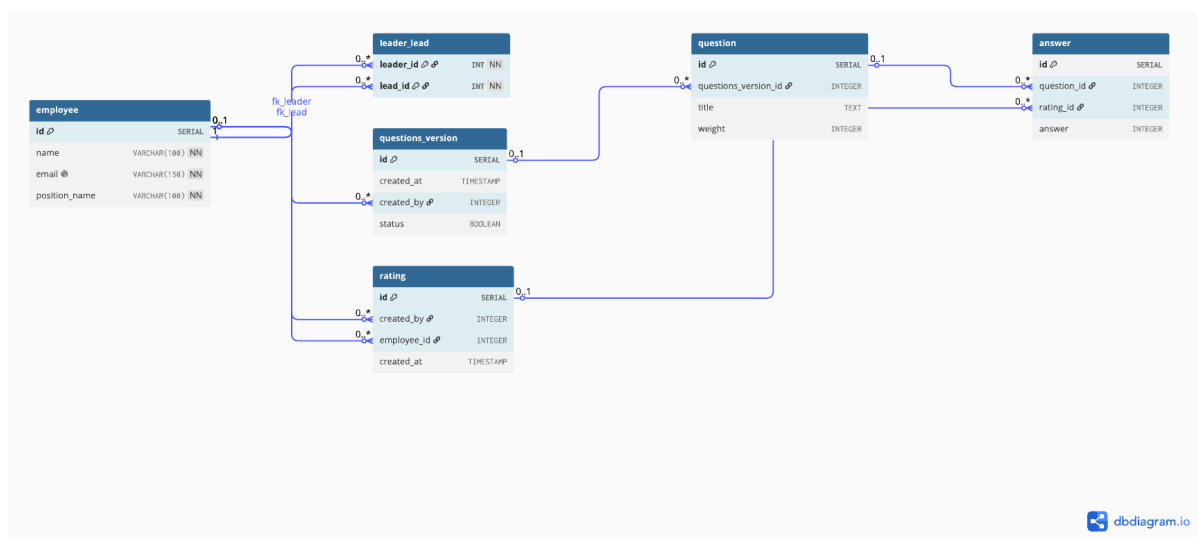


Diagrama ER — tabelas de funcionários, liderança, questões e avaliações.

2. Design das telas (protótipo Figma)

Agora, com a estrutura do banco planejada e os requisitos de sistema levantados, segue a elaboração do design do app. Para isso, foi utilizado o FIGMA, software de design de interface.

Seguindo as orientações da case, imaginou-se:

- Um usuário pode ter que avaliar muitos subordinados de uma só vez, portanto o procedimento tem que ser otimizado e sem muitas repetições. (Optei por fazer um modal, simples e rápido, para avaliar diversos funcionários)
- Deve ser claro a quantidade de avaliações a serem feitas/já realizadas em relação a todos os funcionários pelos quais um usuário é responsável, direta ou indiretamente (para que o líder consiga controlar suas avaliações caso exista um prazo para entregá-las)
- Ao inspecionar as avaliações de um funcionário: deve ser possível realizar uma avaliação, e deve existir uma lista de histórico de avaliações com gráfico de desempenho geral e/ou por líder avaliador (visível apenas as avaliações realizadas por subordinados)
- Só é permitida uma avaliação por semana para cada par líder-funcionário
- Listagem de funcionários incluídos na hierarquia e a média da avaliação de cada um

- Login e logout

Portanto, seguem os protótipos das páginas elaboradas no FIGMA (Login, Home dashboard, Funcionários, Modal Avaliação, e Questões):

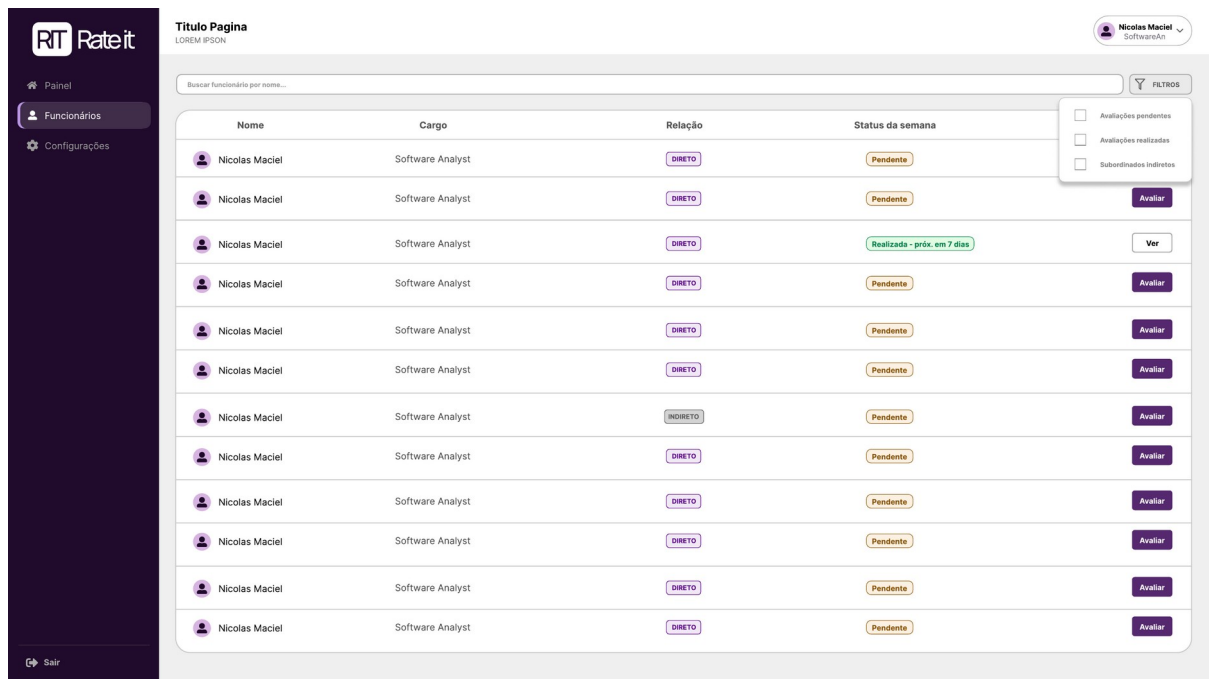
The login form is centered on a light gray background. At the top is the 'RT' logo in a purple square. Below it, the text 'Plataforma de monitoramento de desempenho de funcionários' is displayed. The form is titled 'Entrar como' and features a dropdown menu with 'Nicolas Maciel' and 'Software Analyst' selected. A purple 'Acessar' button is positioned below the dropdown.

Protótipo — Login

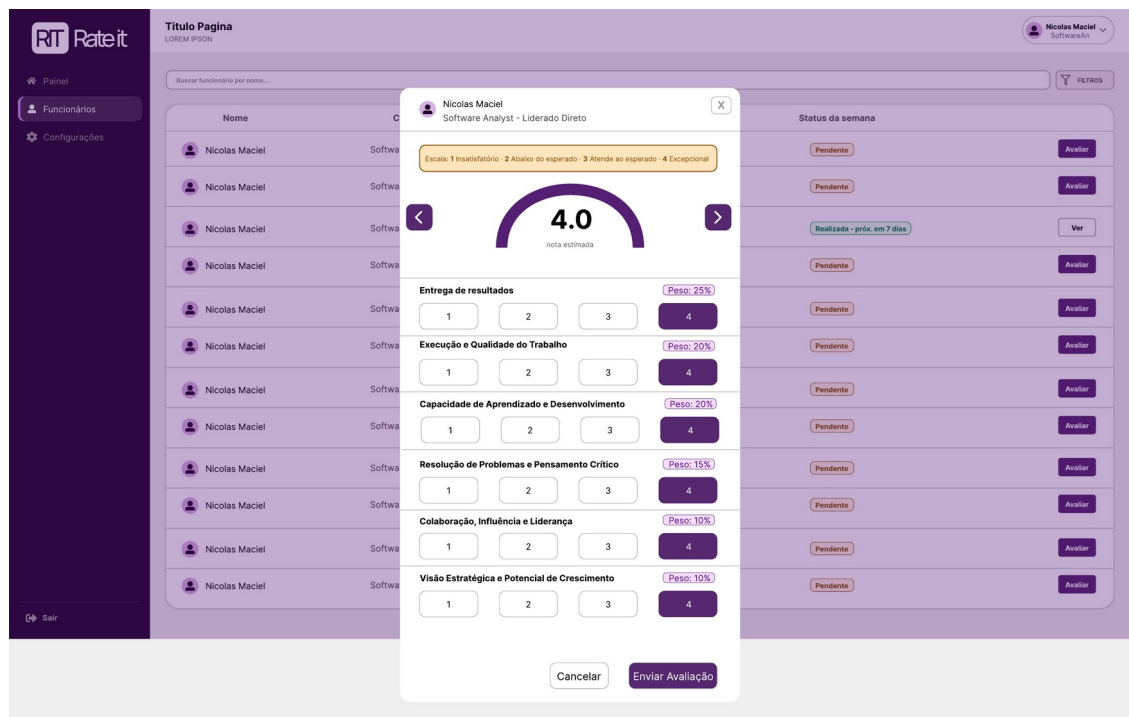
The dashboard is divided into several sections. On the left is a dark purple sidebar with the 'RT Rate it' logo and navigation links: 'Painel', 'Funcionários', and 'Configurações'. The main content area has a header with the page title 'Título Pagina' and a user profile 'Nicolas Maciel Software Analyst'. Below the header, a summary bar shows '8 avaliações pendentes' and a 'Prazo: 1 dia' indicator. The main content is titled 'Olá Nicolas,' and features a table of evaluations. The table has columns for 'Direto', 'Pendente', and 'Avaliar'. To the right of the table are two charts: a bell curve showing 'Non-performers - 10%', 'Average - 70%', and 'Top - 20%', and a bar chart showing performance over months from October to August.

Funcionário	DIRETO	Pendente	Avaliar
Nicolas Maciel Software Analyst			
Nicolas Maciel Software Analyst			
Nicolas Maciel Software Analyst			
Nicolas Maciel Software Analyst			
Nicolas Maciel Software Analyst			
Nicolas Maciel Software Analyst			
Nicolas Maciel Software Analyst			
Nicolas Maciel Software Analyst			

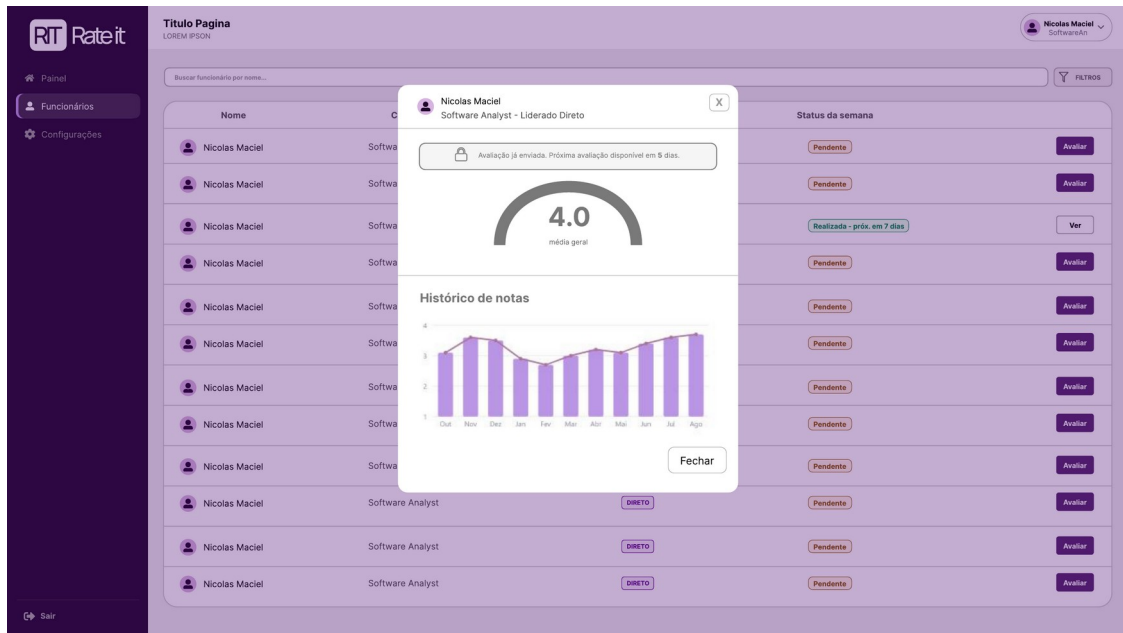
Protótipo — Painel (Dashboard)



Protótipo — Funcionários (listagem e filtros)



Protótipo — Modal de Avaliação (preenchimento)



Protótipo — Modal de Avaliação (já avaliado na semana)

The screenshot shows the 'Tela de Questões' (Questions Screen) in the Rate it application. It displays a table for configuring evaluation questions and weights. The table has columns for 'Título da Questão' (Question Title) and 'Peso (%)' (Weight (%)). The questions are 'Entrega de Resultados' (Delivery of Results) with weights of 25%, 20%, 20%, 15%, 10%, and 10%. The total weight is 100%. A button '+ Adicionar questão' (Add question) is visible. A 'Salvar Alteração' (Save Change) button is at the bottom right.

Protótipo — Tela de Questões

3. Stack tecnológica e desenvolvimento do backend

Ainda precisávamos decidir algumas coisas antes de partir ao desenvolvimento.

A case solicita o uso de frameworks e proficiência nas tecnologias escolhidas. Portanto, a escolha foi:

- Front-end: Vue.js, Pinia, Tailwind
- Back-end: Python, Django Rest Framework
- Infraestrutura: Docker, Nginx
- Banco de dados: Postgres

O Django utiliza serializers e views para os endpoints, o que torna seguro contra injeção de queries e auxilia na validação dos campos de entrada. Além disso, torna fácil a implementação de uma forma de "autenticação" que imaginei.

No desenvolvimento, comecei criando os models das tabelas no ORM do Django.

OBS: Por motivos de organização, separei em dois serviços dentro do app: `table_administration_service` (cadastros e configurações) e `evaluation_service` (regras e cálculos). Dessa forma, a manutenção futura se torna mais intuitiva e simples.

No momento, como não havia criado ainda o ambiente docker, utilizei o banco SQLite nativo do Django para a realização dos testes.

O desenvolvimento foi guiado por testes (TDD), resultando ao todo em 19 testes unitários e de integração (regras de segurança, endpoints de criação e retorno de dados).

Foi necessário um cuidado maior na criação da view da lista de funcionários do painel, pois ela precisa conectar múltiplas tabelas e informações, o que poderia causar lentidão pela quantidade de queries no banco (Big O notation). É justamente por isso que utilizamos a CTE recursiva.

A CTE recursiva se resume na criação de um script SQL que varre a árvore de hierarquia de forma otimizada.

Também tive a ideia de mostrar o tempo restante para a próxima avaliação de cada usuário na listagem do dashboard. Para evitar o mesmo problema de múltiplos cálculos e queries simultâneas, adicionei na view do dashboard uma query única de carregamento dos tempos de avaliação dos funcionários, tornando fácil o cálculo dos dias remanescentes.

Antes de partir para o desenvolvimento do frontend, optei por configurar o ambiente docker, pois, ao tentar conectar a API com o site, teríamos um problema de CORS.

Portanto, comecei criando os containers do banco e do Django.

OBS: O case informa para passar env com chaves e secrets, mas, para evitar problemas e complicações, as chaves do banco estão diretamente no código do docker-compose. Dessa forma, apenas ao rodar o docker, tudo já estará funcionando e pronto para utilizar.

4. Desenvolvimento do frontend

Com o ambiente docker configurado e a API respondendo sem problemas de CORS, parti para o desenvolvimento do frontend, utilizando Vue.js, Pinia e Tailwind, como já mencionado.

Autenticação (X-User-Email)

A própria case explica que não seria necessário um login completo, mas sim uma forma simples de identificar qual líder está fazendo a avaliação, com uma maneira fácil de trocar entre diferentes líderes. Por isso, optei por não fazer um sistema de login tradicional (com senha e token), e sim algo mais simples: o usuário informa o e-mail dele (que precisa já estar cadastrado no banco) e esse e-mail passa a identificar quem está usando o sistema em todas as ações seguintes, através de um cabeçalho (header) chamado X-User-Email, que vai junto em cada pedido feito para a API.

Para isso, criei um lugar central no Pinia (chamado auth) que guarda os dados do usuário logado. Quando alguém digita o e-mail na tela de login, o sistema confirma se ele existe no banco e, se sim, guarda o nome,

cargo e e-mail desse funcionário. Esses dados também ficam salvos no localStorage do navegador, para que o usuário não perca o "login" ao atualizar a página.

Para não precisar ficar adicionando esse e-mail manualmente em cada tela, configurei o sistema para que, toda vez que o front pedir alguma informação para a API, ele já mande o e-mail salvo automaticamente junto, sem eu precisar repetir esse código em cada tela nova que criar.

Também fiz com que, se o usuário tentar acessar alguma página do sistema sem estar "logado", ele seja levado direto para a tela de login. E o contrário também acontece: se ele já estiver logado e tentar voltar para a tela de login, vai direto para o painel.

Como trocar de líder facilmente era importante (tanto para testar o sistema quanto para situações reais, como alguém cobrindo temporariamente outro líder), o botão de sair simplesmente limpa esses dados salvos e devolve o usuário para a tela de login, onde ele pode entrar com outro e-mail cadastrado.

OBS: Se comparar o resultado do front final com o protótipo, vai perceber elementos que não foram incluídos — por demanda de tempo, optei por fazer um front mais simplificado.

OBS: Ainda que a case não peça controle de acesso por cargo, deixei a tela de Questões (onde dá para mudar as perguntas e pesos da avaliação) visível apenas para os cargos mais altos (CEO, CTO, CFO), verificando o cargo salvo do usuário logado. Adicionei a restrição no backend também posteriormente, por isso não comentei na sessão desenvolvimento acima.

Painel (Dashboard)

A case pede que fique claro para o líder quantas avaliações estão pendentes e quantas já foram feitas, olhando para toda a sua hierarquia (diretos e indiretos), principalmente quando existe um prazo para entregar. Por isso, logo no topo da Home coloquei um resumo: quantas avaliações estão pendentes, quantas são diretas, quantas indiretas, e quanto tempo falta para o prazo.

Esse resumo vem pronto de uma única busca na API, que já aproveita a mesma lógica da árvore de hierarquia feita no banco (a consulta recursiva comentada na parte do backend): o backend varre quem é liderado direto e indireto, conta quem falta avaliar, e devolve isso pronto para o front. O front só olha o tamanho dessa lista e mostra na tela, sem precisar calcular nada sozinho.

Logo abaixo, a lista de avaliações permite filtrar entre Pendentes, Realizados e Todas, para o usuário não precisar ir até a tela de Funcionários só para saber o que falta fazer.

Também coloquei dois gráficos: um Progresso das avaliações da semana atual (o total de avaliações concluídas e pendentes no ciclo de uma semana) e outro mostrando a média geral da equipe mês a mês. Para desenhar esses gráficos usei uma biblioteca chamada Chart.js. Os dados de ambos também vêm prontos do backend — o front só recebe os números já calculados e desenha o gráfico em cima deles.

Isso não é um requisito da case, mas achei que fazia sentido para um sistema de "monitoramento de desempenho", que é justamente a proposta do produto criado (o RIT). O layout segue o protótipo do Painel mostrado anteriormente.

Funcionários

Essa tela atende o requisito de mostrar todos os funcionários da hierarquia com a média de avaliação de cada um. Como já falado na parte do backend, essa lista depende da consulta recursiva (CTE) para montar a árvore de liderados diretos e indiretos de forma rápida — o front pede essa lista completa para a API uma única vez e a guarda.

Por causa disso, a busca por nome e os filtros (pendentes, realizadas, subordinados indiretos) não fazem um pedido novo para a API a cada letra digitada ou clique: eles apenas escondem ou mostram, na hora, os funcionários que já foram carregados, deixando a busca instantânea mesmo em hierarquias grandes.

Cada linha mostra se a relação é direta ou indireta, e o status da semana (pendente, ou já realizada, mostrando quanto tempo falta para a próxima avaliação liberar) — essa informação também já vem dentro da mesma lista que a API devolve, sem precisar de uma busca separada para cada funcionário. O layout segue o protótipo de Funcionários mostrado anteriormente.

Modal de Avaliação

Como a case deixa claro que um líder pode precisar avaliar vários subordinados de uma vez, optei por fazer isso em um modal (uma janela que abre por cima da tela) em vez de uma página separada — assim o usuário não perde o lugar onde estava na lista, avalia e volta para o mesmo ponto.

O modal mostra a escala de resposta explicada (de 1, Insatisfatório, até 4, Excepcional) e as perguntas cadastradas, cada uma com o seu peso (ex: 25%) para orientar o avaliador. A nota final, que junta o peso de cada pergunta com a resposta dada, é calculada pelo backend no momento em que a avaliação é enviada — o front só mostra os pesos durante o preenchimento, para não arriscar mostrar uma nota "prévia" que depois pudesse não bater com a nota real (diferente do protótipo original, que já mostrava uma nota estimada em tempo real — simplifiquei essa parte por demanda de tempo).

Uma regra importante da case é que, depois de enviada, a avaliação não pode ser alterada, e só é permitida uma nova avaliação por semana para cada par líder-funcionário. Para isso, assim que o usuário abre o modal de alguém, o front pergunta rapidamente à API se aquele par líder-funcionário já foi avaliado nessa semana. Se a resposta for "sim", o modal já abre no modo de visualização: mostra a nota que já foi dada, quanto tempo falta para poder avaliar de novo, e um gráfico com o histórico de notas daquele funcionário — tudo isso já vem junto nessa mesma resposta da API.

Também vale lembrar a regra de que um líder só pode ver avaliações que ele mesmo fez aos seus subordinados (não vê a própria avaliação, nem a dos colegas ou superiores). Isso é resolvido inteiramente no backend — a busca já retorna só o que aquele e-mail tem permissão de ver, então o front só mostra o que a API devolve, sem precisar esconder nada por conta própria. O comportamento segue os dois protótipos do modal mostrados anteriormente (preenchimento e já avaliado).

Tela de Questões

Essa tela existe para cumprir o requisito de deixar as perguntas da avaliação editáveis com o tempo, sem perder o histórico — foi por isso que criei a tabela `questions_version` no backend. Toda vez que uma pergunta ou peso muda e o usuário salva, isso não edita as perguntas antigas: cria uma versão nova, e as avaliações que já foram feitas continuam ligadas à versão que existia na época, para nunca bagunçar relatórios antigos.

No front, essa tela permite editar o título e o peso de cada pergunta, adicionar novas perguntas, e vai somando os pesos em tempo real: se a soma não fechar em 100%, o botão de salvar fica desativado e aparece um aviso vermelho, para evitar salvar uma configuração errada sem perceber.

Como já contado na parte de autenticação, deixei essa tela visível só para os cargos mais altos (CEO, CTO, CFO). Se alguém de outro cargo tentar acessar, a tela some e aparece um aviso de acesso restrito no lugar dos campos de edição. O layout segue o protótipo da tela de Questões mostrado anteriormente.

Conclusão

Obrigado pela atenção! Seguem abaixo os links do projeto no Figma e do repositório GitHub:

Projeto no Figma: <https://www.figma.com/design/lWaHHOTTVsoV4ojSUDGi1G/Untitled?node-id=0-1&t=XYOBgHYoY5ReSecs-1>

Repositório no GitHub: <https://github.com/nickkjj/Rit-App>

Nicolas da Silva Maciel